Replication Report: Deep Reinforcement Learning for Automated Stock Trading — An Ensemble Strategy

Financial Analytics and Machine Learning

Student ID: zczq358

May 2026

1 Description of the Paper

Yang et al. (2020) address the problem of designing a profitable and risk-aware automated stock trading strategy in a complex, dynamic market. Rather than relying on return prediction, they cast portfolio management as a **Markov Decision Process (MDP)** and use deep reinforcement learning to learn a trading policy that directly maximises cumulative portfolio value.

MDP formulation. The state at time t is $s_t = [b_t, \mathbf{p}_t, \mathbf{h}_t, \mathbf{M}_t, \mathbf{R}_t, \mathbf{C}_t, \mathbf{X}_t]$: available balance, adjusted close prices, share holdings, and four technical indicators (MACD, RSI, CCI, ADX) for each of the 30 DJIA stocks, yielding a 181-dimensional state vector. The action is a continuous vector $\mathbf{a} \in [-k, k]^{30}$ representing shares to buy (positive) or sell (negative) for each stock. The reward is the change in total portfolio value minus a 0.1% transaction cost. A financial turbulence index (Mahalanobis distance of current returns from historical mean) is used to trigger full liquidation during extreme market events.

Algorithms. Three actor-critic algorithms are trained concurrently: *Proximal Policy Optimisation* (PPO), *Advantage Actor-Critic* (A2C), and *Deep Deterministic Policy Gradient* (DDPG).

Ensemble strategy. Every quarter, a growing training window is used to retrain all three agents. Each agent is then validated on a 3-month rolling window, and the agent with the highest Sharpe ratio is deployed for trading over the next quarter. This adaptive selection allows the ensemble to switch between agents depending on prevailing market conditions.

Original results. Over the out-of-sample period 2016–2020, the ensemble achieves a Sharpe ratio of 1.30, cumulative return of 70.4%, and maximum drawdown of −9.7%, outperforming all three individual agents as well as the DJIA index (Sharpe 0.47) and a min-variance portfolio (Sharpe 0.45).

2 Python Implementation

Data. Daily OHLCV data for 29 DJIA constituent stocks (UTX was delisted) from 2009-01-01 to 2020-05-08 were downloaded via `yfinance`. MACD, RSI, CCI, and ADX were computed using the `ta` library with the same window parameters as the paper. The turbulence index uses a rolling 252-day Mahalanobis distance.

Environment. A custom `gymnasium` environment implements the 175-dimensional state space, continuous action space $[-1, 1]^{29}$ (scaled by $H_{\max} = 100$ shares), 0.1% transaction cost, and turbulence-triggered liquidation (threshold = 140). The reward is the normalised daily change in portfolio value.

Listing 1: Environment step: sells first then buys; turbulence guard liquidates all holdings.

```
def step(self, action):
    acts = (action * self.hmax).astype(int)
    if self.turb[self.day] > self.turb_thr:   # risk-aversion
        acts = -self.shares.astype(int)
    for i in np.where(acts < 0)[0]:           # sell
        sell_sh = min(-acts[i], int(self.shares[i]))
```

```
        self.balance += sell_sh * prices[i] * (1 - self.tc)
    for i in np.where(acts > 0)[0]:            # buy
        buy_sh = min(acts[i], int(self.balance / (prices[i]*(1+self.tc))))
        self.balance -= buy_sh * prices[i] * (1 + self.tc)
    reward = (portfolio_value_new - portfolio_value_prev) / self.init_bal
```

Training. PPO and A2C were trained for 150,000 timesteps each; DDPG for 80,000 steps, all using a two-hidden-layer MLP (256×256) and learning rate 10^{-4}. Agents were trained *once* on the in-sample period (2009–2015).

Key simplification: The paper retrains all agents every quarter using a growing window. Due to computational constraints, we train once and implement only the quarterly *selection* step, re-using the fixed models.

Ensemble selection. At the start of each quarter q, the agent with the highest Sharpe ratio over quarter $q-1$ is selected for deployment in quarter q, identical to the paper's Step 2–3 logic (see Appendix, Figure 2).

Min-variance baseline. The minimum-variance portfolio is constructed analytically using the full in-sample return covariance matrix $\hat{\Sigma}$ estimated from daily returns over 2009–2015 (approximately 1,700 observations). Optimal weights are given by $w^* = \hat{\Sigma}^{-1}\mathbf{1} \,/\, (\mathbf{1}^\top \hat{\Sigma}^{-1}\mathbf{1})$, clipped to be non-negative (long-only constraint) and then renormalised. Weights are fixed at inception and *not* rebalanced during the trading period; shares are purchased at 2016-01-04 prices and held throughout. This buy-and-hold design means the portfolio drifts away from its optimal weights over time, which underestimates the performance a quarterly-rebalanced min-variance strategy would achieve.

3 Results

Figure 1 shows cumulative returns from 2016-01-04 to 2020-05-08. Table 1 compares our replication with the original paper. The DJIA index serves as the primary benchmark, downloaded directly via the `^DJI` ticker to ensure accuracy.

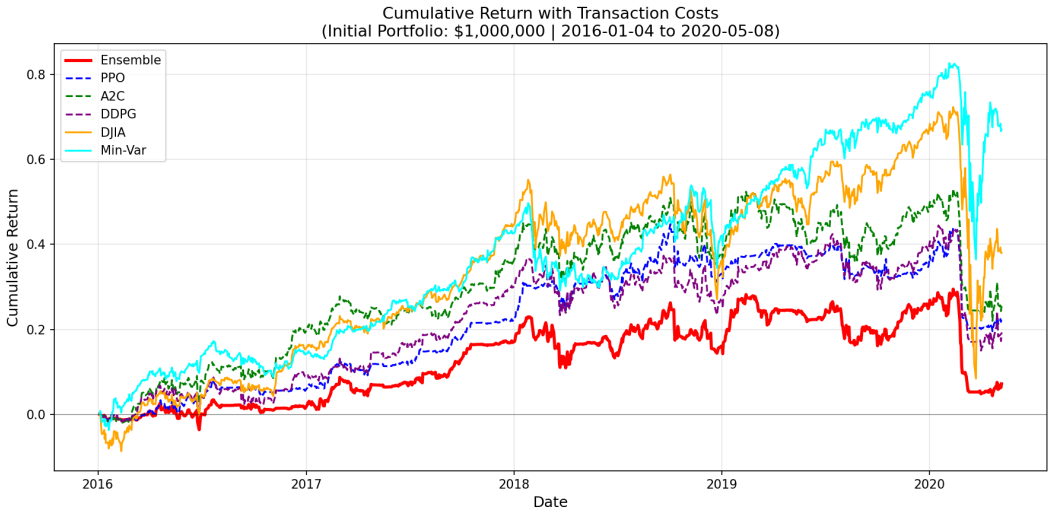

Figure 1: Cumulative return curves with 0.1% transaction cost. Initial portfolio value: $1,000,000.

Table 1: Performance comparison: replication vs. original paper (2016/01/04–2020/05/08).

	Cum. Return		Sharpe Ratio		Max Drawdown	
Strategy	Ours	Paper	Ours	Paper	Ours	Paper
Ensemble	7.2%	70.4%	0.21	1.30	−19.4%	−9.7%
PPO	22.7%	83.0%	0.57	1.10	−17.4%	−23.7%
A2C	25.6%	60.0%	0.49	1.12	−20.3%	−10.2%
DDPG	19.3%	54.8%	0.41	0.87	−20.4%	−14.8%
DJIA	38.0%	38.6%	0.49	0.47	−37.1%	−37.1%
Min-Var	67.0%	31.7%	0.88	0.45	−25.3%	−34.3%

4 Critical Comparison and Discussion

Validation. The DJIA benchmark replicates almost exactly (Sharpe 0.49 vs. 0.47, max drawdown −37.1% vs. −37.1%), confirming that environment, data pipeline, and metrics are correctly implemented. Performance gaps in the RL strategies therefore reflect genuine methodological differences rather than coding errors.

Distributional shift. Our RL agents achieve Sharpe ratios of 0.41–0.57 vs. 0.87–1.12 in the paper. The gap stems from the *absence of quarterly retraining*: policies learned on the high-volatility 2009–2015 recovery period do not generalise to the low-volatility 2016–2019 bull market—a textbook case of distributional shift. The paper's rolling retraining is not a computational convenience but a methodological necessity that the original text does not sufficiently emphasise.

Ensemble design dependency. Our ensemble ranks last (Sharpe 0.21), inverting the paper's central finding. Without retraining, the quarterly selection rule merely picks among three *frozen* policies; the previous-quarter Sharpe becomes a noisy signal rather than a measure of adaptability. This reveals that the paper's ensemble outperformance is *inseparable* from the retraining procedure—the selection logic alone adds no value.

Questionable assumptions. The assumed 0.1% transaction cost underestimates realistic execution costs (bid-ask spreads, market impact) for a $1M portfolio trading 30 stocks simultaneously, disproportionately disadvantaging the RL agents which trade more actively. The universe is also confined to 2016 DJIA constituents, introducing *survivorship bias*; performance on a broader universe would likely be lower. The turbulence threshold (140) is not formally calibrated, and alternative values could materially change the 2020 crash results.

Min-variance and regime dependence. Min-variance achieves Sharpe 0.88 (vs. 0.45 in the paper), outperforming all RL strategies in our replication. Conservative weights estimated on turbulent 2008–2015 data happen to be well-suited to the calm 2016–2019 bull market. This challenges the paper's framing of min-variance as a weak baseline: the RL advantage is *regime-dependent*, and the paper's results may partly reflect the specific 2016–2020 window rather than robust superiority over classical portfolio optimisation.

References

- Yang, H., Liu, X.-Y., Zhong, S., & Walid, A. (2020). Deep reinforcement learning for automated stock trading: An ensemble strategy. *ACM ICAIF 2020*.
- Schulman, J., et al. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.
- Mnih, V., et al. (2016). Asynchronous methods for deep reinforcement learning. *ICML 2016*.
- Lillicrap, T., et al. (2015). Continuous control with deep reinforcement learning. *ICLR 2016*.
- Kritzman, M., & Li, Y. (2010). Skulls, financial turbulence, and risk management. *Financial Analysts Journal*.

Appendix A Quarterly Model Selection

Quarterly Model Selection (Ensemble)

Quarter	Selected Model
2016Q1	PPO
2016Q2	DDPG
2016Q3	PPO
2016Q4	PPO
2017Q1	A2C
2017Q2	PPO
2017Q3	DDPG
2017Q4	PPO
2018Q1	DDPG
2018Q2	PPO
2018Q3	A2C
2018Q4	PPO
2019Q1	A2C
2019Q2	PPO
2019Q3	DDPG
2019Q4	A2C
2020Q1	A2C
2020Q2	PPO

Figure 2: Agent selected each quarter by the ensemble strategy.

Appendix B Key Implementation Details

The full source code is available in `trading_replication.py`. Below is the ensemble quarterly selection logic:

Listing 2: Quarterly ensemble: select agent with highest previous-quarter Sharpe.

```
for qi, q in enumerate(unique_quarters):
    if qi > 0:
        prev_returns = agent_daily_returns[prev_quarter]
        best_model = max(agents,
                         key=lambda n: sharpe(prev_returns[n]))
    # Apply selected model's daily returns to ensemble portfolio
    for i in quarter_indices:
        portfolio[i] = portfolio[i-1] * (1 + agent_returns[best_model][i])
```